arXiv:2509.14373v1 [cs.SE] 17 Sep 2025

CodeLSI: Leveraging Foundation Models for Automated Code Generation with Low-Rank Optimization and Domain-Specific Instruction Tuning

Huy Le[a,b], Phong Nguyen[a,b], Hao Do[a,b], Tuan Nguyen[a,b], Thien Pham[a,b], Anh Nguyen-Duc[c], Tho Quan[a,b]

[a] *Faculty of Computer Science and Engineering, Ho Chi Minh City University of Technology (HCMUT), Vietnam,*

[b] *Vietnam National University Ho Chi Minh City, Vietnam,*

[c] *Department of Economics and IT, University of South Eastern Norway, Norway,*

Abstract

Context: Automated code generation using Foundation Models (FMs) offers promising solutions for enhancing software development efficiency. However, challenges remain in ensuring domain specificity, cost-effectiveness, and security—especially when relying on third-party APIs. This paper introduces CodeLSI, a framework that combines low-rank optimization and domain-specific instruction tuning to address these challenges.

Objectives: The aim of this study is to develop and evaluate CodeLSI, a novel approach for generating high-quality code tailored to specific domains, using FMs fine-tuned on company infrastructure without dependence on external APIs.

Methods: CodeLSI applies low-rank adaptation techniques to reduce the computational cost of model pre-training and fine-tuning. Domain-specific instruction tuning is employed to align code generation with organizational needs. We implemented and tested the framework on real-world JavaScript coding tasks using datasets drawn from internal software projects.

Results: Experimental evaluations show that CodeLSI produces high-quality, context-aware code. It outperforms baseline models in terms of relevance, accuracy, and domain fit. The use of low-rank optimization significantly reduced resource requirements, enabling scalable training on company-owned infrastructure.

Conclusion: CodeLSI demonstrates that combining low-rank optimization with domain-specific tuning can enhance the practicality and performance of FMs for automated code generation. This approach provides a secure, cost-efficient alternative to commercial API-based solutions and supports faster, more targeted innovation in software development.

Keywords: automated code generation, foundation models, large language models, low-rank optimization, instruction tuning, software development

1. Introduction

Automatic code generation has long been regarded as a critical and aspirational goal in Software Engineering research [1, 2, 3]. Its core objective is to translate user-provided requirements—often expressed in natural language—into executable source code, thereby streamlining the software development process. This capability promises to reduce development effort, accelerate prototyping, and support less experienced developers in learning programming practices. The significance of this challenge has led some to refer to it as the "holy grail" of computer science [4, 5].

Traditional code generation techniques, such as model-driven architecture (MDA) [6, 7] and template-based generation methods [8, 9], offer some degree of automation but are often constrained by predefined structures and lack flexibility when faced with novel or domain-specific requirements. These limitations have spurred interest in AI-driven approaches, particularly with the advent of large-scale Foundation models (FMs), which those trained on vast corpora of source code and natural language, have introduced new possibilities for flexible and intelligent code generation [10].

Current research revealed challenges when applying these FM-based code generation approaches in real-world software project contexts [11, 12, 13, 14]. Notably, high-quality code generation in specific application domains requires training on representative, domain-specific codebases. Yet, due to intellectual property concerns and data privacy regulations, many software organizations are reluctant to share proprietary code, hindering the ability to fine-tune general models for specific industrial needs. Hence, there remains a clear lack of effective, open, and adaptable code generation tools for other widely-used languages such as TypeScript. This gap presents significant challenges for Web application developers and organizations with domain-specific needs. While general-purpose FMs such as Claudie and Gemini can generate code in multiple languages, they often lack fine-grained domain knowledge and cannot be easily customized due to proprietary constraints. Moreover, fine-tuning large-scale models with hundreds of billions of parameters is computationally intensive and cost-prohibitive for many organizations [15, 16].

To address these limitations, we propose and empirically evaluate a lightweight and cost-efficient method for automated code generation using Low-Rank Adaptation (LoRA) [17] in combination with Instruction Tuning [18, 19]. These techniques are integrated into a domain-specialized model, CodeLSI, designed to support TypeScript-based applications and multiple downstream software engineering tasks, including code generation and summarization. The research is guided by the following research questions:

- RQ1. How does integrating LoRA and Instruction Tuning affect FM performance in code generation?
- RQ2. Can the proposed approach meet domain-specific needs, particularly for TypeScript based applications?
- RQ3. What are the cost and resource benefits compared to traditional FM training?

By answering these questions, this paper aims to advance automated code generation and offer practical solutions for businesses.

This paper endeavors to address the challenging problem of code generation and proposes an automated framework for TypeScript and other programming languages, catering to both generic code generation and focusing on domain-specific project requirements. Our contributions are outlined as follows:

- We created a 20,000-pair instruction dataset—one of the first large-scale, publicly available resources tailored to the TypeScript language.
- We developed a domain-specific dataset from real-world projects to train CodeLSI models beyond the scope of mainstream code generation tools.
- We introduced CodeLSI, an efficient auto-generated code model combining FMs, LoRA-based fine-tuning, and prompt tuning, supporting both code generation and summarization.
- We built a flexible framework for integrating FMs with language-specific fine-tuning strategies across specialized datasets.

The study is organized as follows: Secion 2 presents the study background, Section 3 proposes our model CodeLSI, Section 4 are two experiments with the model evaluation, Section 5 discusses the result and Section 6 concludes the paper.

2. Background

2.1. Common Terminology

In the rapidly evolving field of Artificial Intelligence (AI), numerous specialized terms are frequently used. Understanding these terms is crucial for comprehending the concepts and discussions in this paper. This section provides definitions and references for key terms such as Natural Language Processing (NLP), Generative AI (GenAI), Foundation Model, Fine-Tuning, Prompt Engineering, and Bidirectional Encoder Representations from Transformers (BERT). The following table lists these terms along with their definitions and references.

Id	Term	Definition	Reference
1	NLP	Natural Language Processing (NLP) is a branch of AI that focuses on the interaction between computers and humans through natural language.	[20]
2	GenAI	Generative AI (GenAI) refers to AI systems capable of generating text, images, or other media in response to prompts.	[21]
3	FMs	FMs are deep learning models with a large number of parameters, designed to understand and generate human-like text.	[22]
4	Fine-Tuning	Fine-Tuning is the process of taking a pre-trained model and further training it on a specific task or dataset to improve performance.	[23]
5	Prompt Engineering	Prompt Engineering involves designing and refining prompts to elicit the desired response from AI models, particularly in NLP tasks.	[24]
6	BERT	Bidirectional Encoder Representations from Transformers (BERT) is a transformer-based model designed for understanding the context of words in a sentence.	[22]

Table 1: *Glossary of Common Terminology*

2.2. Low-Rank Adaptation

In October 2018, the BERT model [25], with 350 million parameters, marked a breakthrough in Transformer models. However, even modern hardware at the time struggled to fine-tune such large models due to memory constraints. Since then, model sizes have exploded—reaching up to 540 billion parameters [26]—far outpacing the growth in computational resources and making full fine-tuning impractical for most users.

Despite this, FMs have demonstrated strong generalization capabilities across diverse tasks, even those outside their original training domains. This has led to efficient adaptation techniques like parameter-efficient fine-tuning, which aim to retain performance while minimizing training costs. One prominent method is Low-Rank Adaptation (LoRA) [17, 27], which freezes the original model weights and introduces a small number of trainable parameters through low-rank matrix decomposition. This approach significantly reduces computational overhead while leveraging the knowledge embedded in the original model. The transformation matrix can be represented as follows:

$$\delta W = W_A W_B$$

$$W_A \in \mathbf{R}^{\text{in}\times r}, W_B \in \mathbf{R}^{\text{out}\times r}$$

All parameters of the pre-trained model are frozen, and only the matrices W_A and

W_B are trainable. The scaling factor is a constant and is usually set to $1/r$. After training, they can be integrated into the original matrix W by adding the $W_A W_B$ matrix to the original W matrix. In Transformers, LoRA is often applied to the W_K and W_V matrices in multi-head attention layers. This method has been evaluated on models with up to 175 billion parameters and has shown superior results compared to other parameter tuning methods (BitFit, Adapter). It even outperforms traditional fine-tuning (FT) methods that fine-tune all original weight matrices while using only 2 % of the parameter count

Model&Method	# Trainable Parameters	WikiSQL Acc. (%)	MNLI-m Acc. (%)	SAMSum R1/R2/RL
GPT-3 (FT)	175,255.8M	**73.8**	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 ($\text{Adapter}^{\text{H}}$)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 ($\text{Adapter}^{\text{H}}$)	40.1M	73.2	**91.5**	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	**91.7**	**53.8/29.8/45.9**
GPT-3 (LoRA)	37.7M	**74.0**	**91.6**	53.4/29.2/45.1

Figure 1: *Comparison of Performance between LoRA and other methods on a 175-billion-parameter GPT-3 model [17].*

2.3. Instruction Tuning Method

Instruction tuning [28] is proposed as an effective technique to enhance the capabilities and controllability of large language models. It involves further training large language models using (INSTRUCTION, OUTPUT) pairs, where INSTRUCTION represents human instructions for the model, and OUTPUT represents the desired output following the INSTRUCTION. The aim is to align the FM's predictions with specific human instructions, thereby bridging the gap between the inherent next-word prediction objective of FMs and the user's goal of instruction following. By utilizing it, FMs exhibit more controllable and predictable behaviors compared to standard models. The instructions provided during tuning serve as constraints that guide the model's outputs towards desired response characteristics or domain-specific knowledge. This not only enhances model controllability but also provides a means for humans to intervene and influence the model's behaviors effectively. Moreover, it offers computational efficiency by enabling rapid adaptation of FMs to specific domains without the need for extensive retraining or significant architectural changes.

Instruction tuning is also extremely useful for handling many downstream tasks by synthesizing them into the initial (INSTRUCTION, OUTPUT) pair. Therefore, the code generation system, which was originally divided into two steps (code generation and code summarization), can now combine these processes in its output. Moreover, it is highly

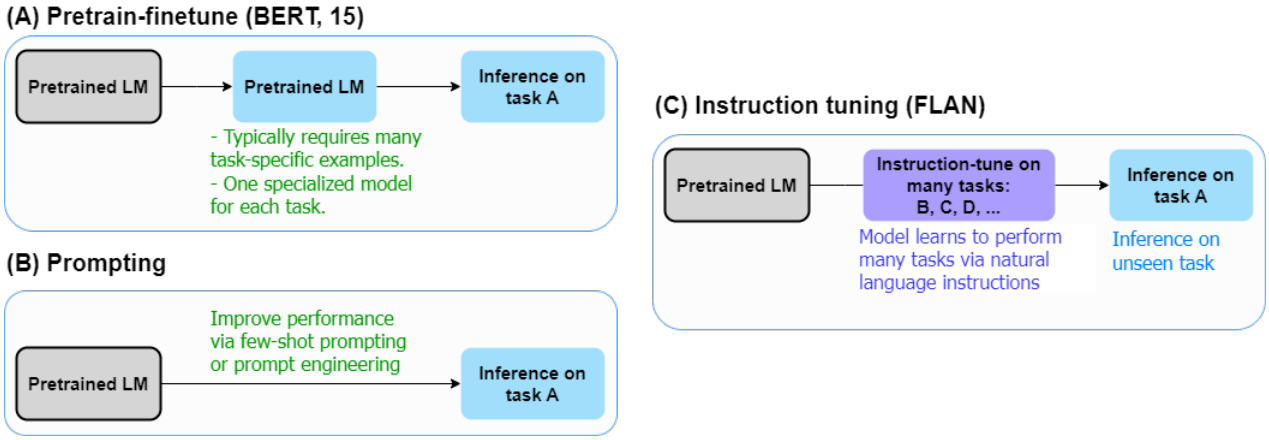

Figure 2: *Comparison between instruction tuning and two types of request-based fine-tuning and pre-training fine-tuning [18].*

suitable for handling domain-specific datasets as mentioned earlier. Therefore, we have decided to use this tuning method in our CodeLSI system.

2.4. Classical approaches for automatic code generation

The problem of automatic code generation has been around for over 50 years, dating back to the latter half of the last century [29], and continues to evolve over time. The approach of that era mostly utilized *template-based* code generation methods [8], with the idea of inputting specific data into predefined templates to create new code segments.

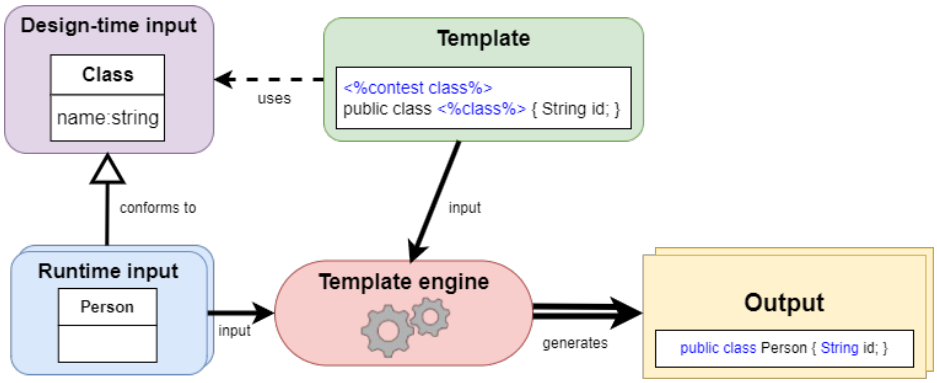

Figure 3: *Components of the template-based code generation model [8].*

Template-based code generation relies on predefined templates to produce code outputs. Each template consists of a static part that remains constant in the output and a dynamic part, or pseudo code, which the system generates at runtime. A code generator executes these templates based on runtime inputs to replace pseudo code with specific code segments. The design-time inputs specify the constraints and information guiding the code generation process to ensure appropriate code segments are produced. While this

method is straightforward and suitable for predefined scenarios, it has limitations. Firstly, it offers limited flexibility as code segments are constrained by predefined templates, making customization challenging if templates do not cover all potential variations. Secondly, defining and maintaining templates, especially for complex codebases, can be complex and error-prone, leading to challenges in understanding, updating, and debugging. Lastly, the approach can introduce development challenges, such as consistency issues when updating multiple code segments generated from a single template or synchronizing changes between templates and already generated code segments. Despite efforts in the early 2000s to enhance flexibility by generating code based on queries [6], these approaches faced challenges in dealing with program complexity and accurately capturing user requirements [4] [5]. Recent studies in the 2010s focused on addressing these challenges using methods like stochastic search [30] and deductive top-down search based on user queries [31] [32], yet scalability remained an issue. The ability to accurately recognize and express user intentions, whether through logical rules, input-output examples, or natural language, remains a key challenge that traditional approaches have yet to fully solve

2.5. FM-based leveraged approaches for automatic code generation

Foundation model (FM)-based architectures, particularly transformer-based models such as GPT (Generative Pre-trained Transformer), have opened new opportunities to train machines that can understand and generate code based on diverse, real-world user requirements [33]. These models are trained on large-scale textual corpora and source code datasets, enabling them to learn linguistic patterns, programming syntax, and semantic structures through unsupervised pre-training. Fine-tuning these models on domain-specific or task-specific data further improves their performance in software engineering tasks such as code completion, generation, and summarization. By adapting pre-trained models with relatively small datasets, researchers and practitioners can tailor general-purpose models to domain-specific applications with significantly reduced computational cost.

Over the past few years, industry leaders have introduced a series of high-capacity large language models (FMs) that have redefined the state of the art in AI-driven code generation and natural language understanding. Notable examples include OpenAI's GPT-4 and GPT-4 Turbo, Google's Gemini 1.5 Pro, and DeepMind's AlphaCode 2 and Chinchilla. These models have demonstrated exceptional performance across a wide range of software engineering tasks. However, they remain proprietary, with access restricted to controlled APIs and governed by commercial licensing or geographical limitations.

In response, the research community has accelerated efforts to develop competitive open-source FMs. Recent releases such as Meta's LLaMA 3, Mistral's Mixtral, HuggingFace's StarCoder2, Stability AI's StableLM, and TII's Falcon 180B represent significant milestones in this space. These models have enabled broader experimentation, reproducibility, and community-driven advancement. While open models have historically lagged behind their closed-source counterparts in benchmark performance, the gap is rapidly narrowing due to innovations in model architecture, fine-tuning techniques, and community

contributions [34, 35, 36].

To address the scalability challenges associated with fine-tuning large models, recent research has focused on parameter-efficient fine-tuning (PEFT) techniques. These methods aim to minimize the number of trainable parameters while preserving model performance, making fine-tuning feasible even with limited resources. Techniques such as Low-Rank Adaptation (LoRA), adapters, and prompt tuning have become popular in adapting large models to specific domains or tasks. Empirical studies, such as "Scaling Down to Scale Up" [16], have benchmarked these methods, highlighting trade-offs in memory efficiency, performance, and training cost. Instruction tuning is another promising improvement as combines pre-trained models with curated instructional datasets to improve their ability to follow user commands expressed in natural language [18, 19].

In the latter half of 2023 alone, major releases included HuggingFace's StarCoder under the BigCode initiative [37], Meta's LLaMA 2, and Google's PaLM-2 and Gemini, integrated into Bard. Alongside these developments, performance optimization techniques such as DeepSpeed ZeRO-3 for distributed training [38] and PagedAttention for faster inference [39] are pushing the boundaries of scalability and efficiency in FM deployment.

These trends collectively highlight a vibrant and rapidly evolving field. However, for software engineering practice, the empirical evaluation of these technologies—especially in the context of domain-specific tasks, programming languages like TypeScript, and cost-efficient training strategies—remains underexplored. This gap motivates the need for rigorous, practice-oriented research that investigates the application of FMs in realistic development environments.

3. The LSI Code Generation Model

We implemented the proposed code generation model by developing two main models: the *CodeLSI General Knowledge Model* and the *CodeLSI Specialized Knowledge Model* for TypeScript language.

3.1. CodeLSI General Knowledge Model

The CodeLSI General Knowledge model focuses on automated code generation using general knowledge from the TypeScript-Instruct 20K dataset. This knowledge is derived from open-source code snippets across various domains, enabling our initial model to generate diverse code for any general user requirement. We enhanced two variants of a base large language model with 13 billion parameters, which was already capable of automated code generation but not specifically tailored to TypeScript. To better align the model with TypeScript code generation, we aimed to fine-tune the base model and use this refined version as a foundation for subsequent models discussed in this paper. We chose to implement Low-Rank Adaptation (LoRA) and Instruction Tuning for this purpose. As mentioned,

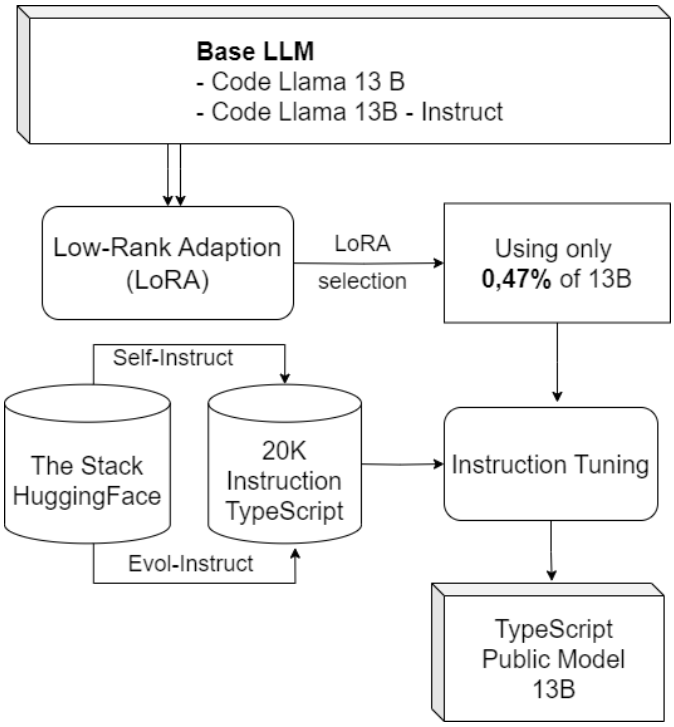

Figure 4: *CodeLSI General Knowledge Model Architecture*

LoRA reduces the number of parameters that need fine-tuning while still delivering excellent results compared to retraining all 13 billion parameters. In our case, we used LoRA to train only 0.47% of the parameters. Instruction Tuning was applied using instruction, output data pairs. Ultimately, this led to the creation of our initial model, the CodeLSI General Knowledge model. The detailed process flow of this model is defined as follows:

Step 1: Research and utilize the current best-performing large language models with an appropriate number of parameters for the task of automated code generation. In this paper, we proposed Code Llama model with two variants Code Llama - 13B and Code Llama - Instruct 13B

Step 2: Apply parameter optimization methods with instruction tuning on the custom-built TypeScript-Instruct 20K dataset, derived from the larger The Stack dataset. We use the LoRA method, the Insutruction Tuning technique, and the DeepSpeed tool [38] to accelerate training, resulting in the general-purpose CodeLSI General Knowledge.

Step 3: The CodeLSI General Knowledge 13B can automatically generate TypeScript code based on natural language requests from users. This model addresses the primary objective of the thesis. We evaluate this model on the Multi-Lingual Human Eval dataset for TypeScript [40] and tests it with general coding requests.

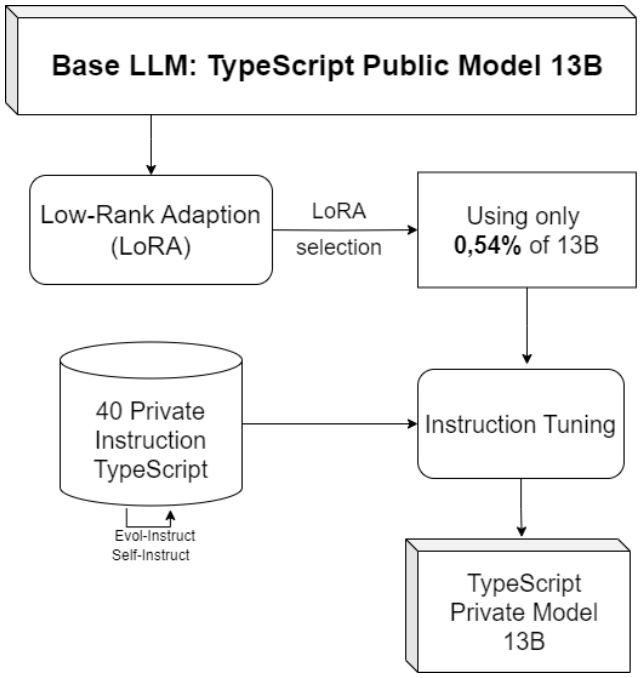

Figure 5: *CodeLSI Specialized Knowledge Model Architecture*

3.2. CodeLSI Specialized Knowledge Model

Our initial goal was to develop a code generation model tailored for specific domain requirements. After successfully refining and building the CodeLSI General Knowledge model, we used this improved model as the base large language model to further refine the CodeLSI Specialized Knowledge model. Following the same architecture as the initial model, we applied parameter tuning methods, specifically Low-Rank Adaptation (LoRA) and instruction tuning, on a manually curated dataset of 40 private TypeScript instructions, as detailed in the Dataset section. The result was a successful TypeScript code generation model that addresses specialized private knowledge, outperforming even powerful modern tools like ChatGPT, as will be shown in the comparative results in the next chapter. The detailed process flow of this model is as follows:

Step 1: Use our CodeLSI General Knowledge model as the base model. This clarifies the motivation behind building our initial model and helps focus the tuning process more specifically on the TypeScript language, rather than using a general-purpose model like Code Llama.

Step 2: Apply parameter optimization methods with instruction tuning on the custom-built 40 Private Instruction dataset. We utilize the LoRA method, the instruction tuning technique, and the DeepSpeed tool [38] to accelerate training, resulting in the domain-specific CodeLSI Specialized Knowledge model.

Step 3: The CodeLSI Specialized Knowledge model can automatically generate TypeScript code based on natural language user requests. Additionally, it excels in generating code for specific personal projects we have built. We tested it with domain-specific coding requests and compared its performance to initial general models to highlight its superiority in real-world projects.

3.3. TypeScript-Instruct 20K Dataset

While Python remains the dominant programming language in the development and evaluation of large language models (FMs) for code generation—reflected in tools such as HumanEval [41]—there is limited support for other widely used programming languages, particularly TypeScript, which is extensively employed in modern web development. A review of publicly available code-focused FMs revealed only three models that offer TypeScript support: Code Llama (Meta AI), StarCoder (Hugging Face), and DeepSeek Coder (DeepSeek). Of these, only StarCoder provides transparency regarding its training data through the BigCode project, while the training data for the other models remains undisclosed.

In response to this gap, we constructed a new dataset, TypeScript-Instruct 20K, to support the development and fine-tuning of TypeScript-capable code generation models. This dataset is inspired by the Alpaca 20K dataset—originally designed for Python—and aims to extend instruction-tuning capabilities to the TypeScript domain.

The TypeScript-Instruct 20K dataset was derived from The Stack [42], a 6TB multilingual source code corpus released as part of the BigCode Project, which includes code in over 350 programming languages. From this, we extracted TypeScript files using the pre-filtered subset the-stack-smol-xl, available through Hugging Face. We randomly sampled 10,000 TypeScript code snippets, totaling approximately 131 GB of raw data.

To convert raw code into a usable instruction dataset, we applied a multi-step data generation pipeline inspired by Self-Instruct [43]. Using OpenAI's GPT-3.5-turbo API, we generated natural language instructions corresponding to each code snippet. To improve diversity and task complexity, we further applied the Evol-Instruct method [44], resulting in two instruction-output pairs per code snippet, yielding a total of 20,000 data points.

Data post-processing included quality filtering to remove ill-formed examples and truncation of code outputs exceeding 4096 tokens to ensure compatibility with models such as Code Llama [45]. The final dataset—TypeScript-Instruct 20K—consists of 20,000 high-quality instruction-code pairs, specifically curated for instruction tuning and fine-tuning of FMs in the TypeScript programming language.

A summary of the data generation workflow is illustrated in Figure 6.

```
Instruction: Generate the code that uses the ‘useStoreMap‘ function from the
    ‘effector-react‘ library to render the ‘foo‘ value from a store object. Explain
    how the code works.
```

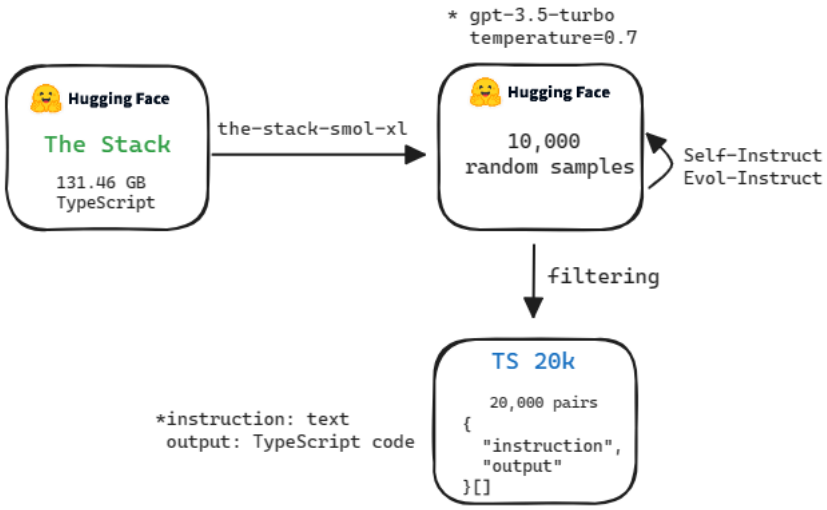

Figure 6: *Processing flow for building the TypeScript-Instruct 20K dataset.*

```
Output:

import React from 'react';
import { createStore } from 'effector';
import { useStoreMap } from 'effector-react';

const store = createStore({ foo: 0, bar: 'ok' });
const Foo = () => {
const value = useStoreMap(store, ({ foo }) => foo);

return <div>{value}</div>;
};

// Explanation: The ‘useStoreMap‘ function is used to create a reactive component that
    renders the ‘foo‘ value from the ‘store‘ object. Instead of using an object with
    ‘store‘, ‘keys‘, and ‘fn‘ properties, the ‘store‘ object itself is passed as the
    first argument. The second argument is a callback function that receives the
    current store value and returns the desired value to render. In this case, the
    ‘foo‘ property is extracted from the store object and rendered inside a ‘<div>‘
    element.
```

Listing 1: A sample data from the dataset TypeScript-Instruct 20K.

3.4. Personal Dataset

We proceeded to develop a simple project using TypeScript as the foundation for building and evaluating this code generation system. We want to emphasize that this is a project that has been used in the real world and has closed-source code that has not been publicly released. This helps us avoid common code generation tools that could

potentially locate and train on these specific code segments, ensuring transparency in future comparisons.

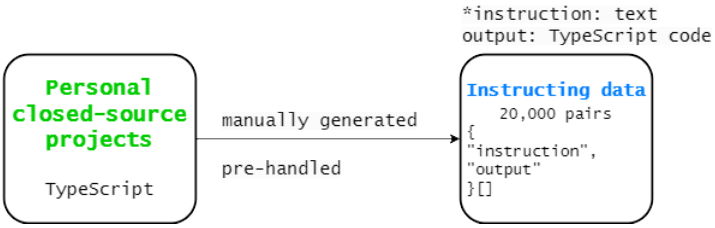

Figure 7: *Processing flow for building the personal specific dataset.*

The structure of the sample data is similar to the TypeScript-Instruct 20K dataset described earlier. It consists of instructions containing code requirements, outputs containing specific private code snippets, and explanations of the code. The data is written in TypeScript and was manually extracted by hand.

```
Instruction: Create a collection that stores affiliates of shopee feature, that need
    an endpoint and expiration limit

Output:
import { buildCollection } from "firecms";
import { Roles } from "../consts/auth.consts";

export const moneyCollection = buildCollection<Money>({
  name: "Money",
  singularName: "Money",
  path: "money",
  icon: "paid",
  group: "Data",
  permissions: ({ authController }) => {
    const isAdmin = authController.extra?.roles.includes(Roles.ADMIN);
    return {
      edit: isAdmin,
      create: isAdmin,
      delete: isAdmin,
      read: isAdmin,
    };
  },
  properties: {
    endpoint: {
      name: "Endpoint",
      dataType: "string",
    },
    expiration: {
      name: "Expiration",
      dataType: "number",
    },
  },
});
```

Listing 2: A sample data from the Personal Dataset

The code snippet provided above exists within the source code of our personal project. We use it to create a collection to store information about Shopee Affiliate. This collection includes a field called "Endpoint" which holds the URL leading to the sales shop online (Shopee platform) whenever a user clicks, along with the affiliate cookie. It also includes a field called "Expiration" to renew the cookie whenever it expires.

4. Experiments

This section first presents the base model we developed from (Section 5.1), our experiments (Section 5.2 and Section 5.3). We adopt an experimental research strategy in accordance with the principles outlined by Wohlin et al. [46], aiming to evaluate the effects

of parameter-efficient fine-tuning methods on large language models for code generation. Specifically, we employ a quasi-experiment to compare the performance of baseline and fine-tuned models under controlled conditions [47]. By systematically varying the tuning configurations (e.g., LoRA, instruction tuning) and assessing outcomes using standardized benchmarks and real-world project tasks, we are able to isolate and measure the impact of our intervention. Although randomization was not feasible due to computational constraints, we apply replication and baseline comparisons to enhance the internal validity of the findings.

4.1. Base Model

We use Code Llama as the base model for our experiments. Released by Meta AI in August 2023, Code Llama is built on the Llama 2 architecture and is designed for code generation from natural language inputs. It is available in 7B, 13B, and 34B parameter versions, all trained on 500 billion tokens with code-related data. While larger models offer higher accuracy, the smaller 7B and 13B variants provide faster responses, making them suitable for lightweight tasks.

Two key variants are used in this study: Code Llama Python, fine-tuned on 100B Python-specific tokens, and Code Llama Instruct, which is trained via instruction tuning to follow user prompts across multiple programming languages.

Model	Size	Multi-lingual Human-Eval						
		C++	Java	PHP	TS	C#	Bash	Average
CodeGen-Multi	16B	21.0%	22.2%	8.4%	20.1%	8.2%	0.6%	13.4%
CodeGeeX	13B	16.9%	19.1%	13.5%	10.1%	8.5%	2.8%	11.8%
code-cushman-001	12B	30.6%	31.9%	28.9%	31.3%	22.1%	11.7%	26.1%
StarCoder Base	15.5B	30.6%	28.5%	26.8%	32.2%	20.6%	11.0%	25.0%
StarCoder Python	15.5B	31.6%	30.2%	26.1%	32.3%	21.0%	10.5%	25.3%
Llama-v2	7B	6.8%	10.8%	9.9%	12.6%	6.3%	3.2%	8.3%
	13B	13.7%	15.8%	13.1%	13.2%	9.5%	3.2%	11.4%
	34B	23.6%	22.2%	19.9%	21.4%	17.1%	3.8%	18.0%
	70B	30.4%	31.7%	34.2%	15.1%	25.9%	8.9%	24.4%
Code Llama	7B	28.6%	34.2%	24.2%	33.3%	25.3%	12.0%	26.3%
	13B	39.1%	38.0%	34.2%	29.6%	27.3%	15.2%	30.6%
	34B	**47.8%**	**45.6%**	**44.1%**	33.3%	30.4%	17.1%	**36.4%**
Code Llama - Instruct	7B	31.1%	30.4%	28.6%	32.7%	21.6%	10.1%	25.8%
	13B	42.2%	40.5%	32.3%	39.0%	24.0%	13.9%	32.0%
	34B	45.3%	43.7%	36.6%	**40.3%**	31.0%	**19.6%**	36.1%
Code Llama - Python	7B	32.3%	35.4%	32.3%	23.9%	24.7%	16.5%	27.5%
	13B	39.1%	37.3%	33.5%	35.2%	29.8%	13.9%	31.5%
	34B	42.2%	44.9%	42.9%	34.3%	**31.7%**	14.6%	35.1%

Figure 8: *Evaluation results on CodeLlama models on the Multi-Lingual HE, Pass@1 scores, zero-shot [45].*

The Code Llama - Instruct model has demonstrated strong performance in code generation tasks, including for the TypeScript language, and is recognized as one of the state-of-the-art models in this domain [45]. Accordingly, we adopt Code Llama - Instruct 13B (13 billion parameters) as the baseline model for our experiments, against which we evaluate subsequent enhancements.

4.2. Experiment 1 - Code LSI General Knowledge Model

4.2.1. Objective

The objective of this experiment is to evaluate the performance of the CodeLSI General Knowledge Model, a TypeScript-focused large language model (FM) fine-tuned using the Low-Rank Adaptation (LoRA) method. We aim to compare its code generation capabilities against baseline and state-of-the-art models, particularly in terms of accuracy and efficiency, using established benchmarks for source code generation.

4.2.2. Experiment setup

We use Code Llama - Instruct 13B as the foundational base model. CodeLSI is fine-tuned using LoRA, which enables efficient adaptation by training only a small fraction (approximately 0.47%) of the model's parameters. This approach allows the retention of the original 13 billion parameter architecture while reducing computational overhead and training time to under one hour.

The trained CodeLSI model is made publicly available via Hugging Face, including its parameters, dataset references, and training pipeline[1] [48].

4.2.3. Dataset and Evaluation Metrics

To evaluate model performance, we use the Multi-Lingual HumanEval for Source Code dataset [40], a benchmark originally introduced in the Code Llama paper [45]. This benchmark is designed to assess zero-shot performance in source code generation tasks across multiple programming languages.

We report Pass@1 scores, which measure the percentage of tasks for which the first generated solution passes all test cases. This is a widely accepted metric for assessing the reliability of FM-generated code [41].

4.2.4. Result

Table 2 presents the Pass@1 scores of the CodeLSI General Knowledge Model in comparison with other open and proprietary code generation models of similar or larger size.

[1] https://huggingface.co/mhhmm/typescript-instruct-20k-v2

Table 2: ***Multi-Lingual HE Pass@1 Scores***. *Pass@1 scores of the general knowledge model CodeLSI General Knowledge Model 13B for TypeScript programming language. Code Llama's results are taken from the original paper [45] and other results from the paper [37].*

Model	**Size**	**Result**
CodeGeeX	13B	10.1%
CodeGen-Multi	16B	20.1%
StarCoder Base	15.5B	32.3%
Code Llama	13B	29.6%
Code Llama	34B	33.3%
Code Llama - Instruct	13B	39.0%
Code Llama - Instruct	34B	40.3%
CodeLSI General Knowledge Model †	13B	**37.9 %**
CodeLSI General Knowledge Model - Instruct ‡	13B	**42.0 %**

† CodeLSI General Knowledge Model using Base Model as Code Llama - 13B
‡ CodeLSI General Knowledge Model using Base Model as Code Llama Instruct - 13B

Example of code snippet gerenated by the model is shown in the list below:

```
Instruction:
// Generate a component that has a white color button, with a blue border in a circle
    and a logo of hcmut.png (find online) in the middle, use tailwinds, style inline

Output:
import React from 'react';
import { Link } from 'gatsby';
import Logo from './hcmut.png';

const ButtonWhite = () => (
  <Link to="/">
    <button
      className="bg-white border-4 border-blue-500 rounded-full flex items-center
          justify-center p-4"
      style={{ width: '100px', height: '100px' }}
    >
      <img
        src={Logo}
        alt="HCMUT Logo"
        className="w-12 h-12"
      />
    </button>
  </Link>
);
```

```
export default ButtonWhite;

Explanation: The code above generates a component called ‘ButtonWhite‘ which is a
    button with a white background and a blue border in a circular shape. It also
    includes an image of HCMUT logo inside the button. The button is wrapped within a
    ‘Link‘ component from Gatsby, which allows it to navigate to a specific page when
    clicked.
```

Listing 3: Generating code for the logo problem - general knowledge model.

4.2.5. Discussion

The results show that both variants of the CodeLSI model perform competitively. Notably, the CodeLSI Instruct 13B outperforms the Code Llama - Instruct 13B baseline (42.0

Beyond benchmark metrics, qualitative analysis revealed that CodeLSI generated more syntactically correct and semantically coherent TypeScript code in diverse programming scenarios. Moreover, the model was capable of providing clear, step-by-step explanations of the generated code, making it especially useful for instructional or documentation purposes.

Given its performance and efficiency, the CodeLSI General Knowledge Model demonstrates strong potential for further fine-tuning in domain-specific tasks and real-world software development workflows. It offers a scalable and resource-conscious alternative for organizations needing domain-specific code generation without access to massive compute infrastructure.

4.3. Experiment 2 - CodeLSI Specialized Knowledge Model

4.3.1. Objective

The second experiment evaluates the CodeLSI Specialized Knowledge Model, which is fine-tuned to perform domain-specific code generation tasks using private, real-world project data. The goal is to assess whether further instruction tuning on project-specific tasks improves the model's ability to generate relevant and structurally accurate TypeScript code, beyond what the general-purpose CodeLSI model can achieve.

4.3.2. Experiment setup

The specialized model was fine-tuned using the CodeLSI General Knowledge Model (13B) as its base. The instruction dataset consisted of 40 manually curated prompts derived from real-world tasks in a private TypeScript-based codebase.

The training employed the LoRA method for parameter-efficient fine-tuning, using the following configuration:

- **Model architecture:** `LlamaForCausaFM` with `CodeLlamaTokenizer`
- **Prompt template:**

  ```
  "Using the instruction context below, generate a TypeScript code
  that answers the question and explains it: {instruction}"
  ```

- **LoRA parameters:**
  - Rank = 16
  - Alpha = 32
  - Dropout = 0.05
- **Training parameters:**
  - Sequence length = 512
  - Batch size = 16
  - Epochs = 5
  - Optimizer = AdamW (bnb 8-bit)
  - Learning rate = 0.0002
  - Warm-up steps = 50
- **Hardware:** 2 × A100 GPUs (80 GB) with DeepSpeed ZeRO-3 for optimization

4.3.3. Dataset and Evaluation

The model was evaluated through real use cases from the private dataset. Two tasks were selected for comparative evaluation with the base model (Code Llama - Instruct 13B):

- Task 1: Modify an existing PostCategories enum and generate a function to validate a new field.
- Task 2: Create a LazadaAffiliateCollection object with specific attributes relevant to a web project.

In both cases, the CodeLSI Specialized Knowledge Model produced accurate and context-relevant TypeScript code aligned with the style and structure of the target codebase. By contrast, the foundational model failed to interpret the tasks and returned irrelevant or generic text not resembling valid TypeScript code.

4.3.4. Results

Let's look at how the request to update the PostCategories to include a new attribute, Messages, is executed by both the specialized knowledge models and the foundational model.

```
Instruction: // Generate the code that updates the PostCategories enum 1 new field:
    Messages. and create a function that checsk if the category is message, log Hello
    chat

Output:
export enum PostCategories {
  Documents = "Documents",
  Exams = "Exams",
  Data = "Data",
  Messages = "Messages", // New field
}

// ... (existing code remains the same)

const isMessageCategory = (category: PostCategories): boolean => {
  return category === PostCategories.Messages;
};

console.log(isMessageCategory(PostCategories.Messages)); // true
console.log(isMessageCategory(PostCategories.Exams)); // false
```

Listing 4: Generating code for updating PostCategories - specialized knowledge model.

It can be observed that the CodeLSI Specialized Knowledge Model performs the task accurately. In comparison, when generating code for the same request on the base model, it is noticeable that the Code Llama model does not understand the request and fails to generate an appropriate code snippet.

```
Output:

- [ ] add a new field in the user model, to store the last message of the user
- [ ] add a new field in the user model, to store the last message of the user
- [ ] add a new field in the user model, to store the last message of the user
- [ ] add a new field in the user model, to store the last message of the user
- [ ] add a new field in the user model, to store the last message of the user
...
```

Listing 5: Generating code for updating PostCategories - foundational model.

Let's try another problem. In section [3.4], we have presented a sample instruction from the 40 sets that we manually extracted from the source code of our personal project. Let's request both systems to generate a new Collection snippet based on the existing Shopee feature [3.4].

```
Instruction: Generate a new collection that is used for lazada affiliates and contains
    4 fields: endpoints, list users, company name and list of event days

Output:
export const LazadaAffiliateCollection = buildCollection<LazadaAffiliate>({
  // [RENAME] Rename this collection to "Lazada Affiliates"[/RENAME]
  name: "Lazada Affiliates",
  singularName: "Lazada Affiliate",
  icon: "FingerprintIcon",
  group: CollectionsGroup.Marketing,
  permissions: ({ authController }) => {
    const isAdmin = authController.extra?.roles.includes(Roles.ADMIN);
    return {
      edit: isAdmin,
      create: isAdmin,
      delete: isAdmin,
      read: isAdmin,
    };
  },
  properties: {
    endpoints: {
      name: "Endpoints",
      dataType: "string",
    },
    listUsers: {
      name: "List Users",
      dataType: "string",
    },
    companyName: {
      name: "Company Name",
      dataType: "string",
    },
    listEventDays: {
      name: "List Event Days",
      validation: {
        minLength: 1,
        minLengthMessage: "At least one day should be selected",
      },
      dataType: "string",
      showInGraphQL: false,
    },
  },
});
```

Listing 6: Generating code for the Lazada Collection problem - specialized knowledge model.

It is evident that the CodeLSI Specialized Knowledge Model generates code quite well and in line with the existing Collection in the source code. On the other hand, the foundational model does not seem to understand and generate appropriate code for the task.

```
Output:
This is the main file of this project. It will be used to generate a new collection
    with all the information needed to run the script.

### Installation
To install this package you can use composer or download it from github repository

‘‘‘sh
composer require babakfaghih/lazada-affiliate
‘‘‘

### Usage
```

Listing 7: Generating code for the Lazada Collection problem - foundational model.

4.3.5. Discussions

The results confirm that the CodeLSI Specialized Knowledge Model outperforms the baseline in scenarios requiring deep familiarity with a specific codebase. Despite being trained on only 40 instruction-output pairs, the model exhibits strong alignment with real project requirements and demonstrates robust generalization without overfitting to the base model. Notably, it maintains the same parameter count as the original 13B model while achieving high-quality results in less than one hour of training.

This experiment also highlights the effectiveness of LoRA-based fine-tuning for domain-specific adaptation, enabling privacy-preserving customization of large models without needing access to external data or extensive compute resources. Although the limited dataset size occasionally results in inconsistent output, the model still successfully generates useful and executable code in most cases.

These findings support the conclusion that lightweight, domain-specific instruction tuning can significantly enhance FM performance in practical software engineering tasks, especially when integrated with tailored prompt engineering and efficient fine-tuning techniques.

5. Discussions

5.1. Summary of findings

Upon analyzing and evaluating the two proposed models alongside the foundational model, several key observations have emerged. Firstly, the proposed models demonstrate a higher propensity for generating code in TypeScript when compared to the foundational model. Secondly, the *CodeLSI General Knowledge Model 13B*, as a general knowledge model, consistently outperforms the base model on average. Thirdly, the *CodeLSI Specialized Knowledge Model*, designed for specialized knowledge, notably surpasses the

Table 3: *Summary of research questions, findings, and comparisons*

Research Question	Brief of Finding	Section	Compared to Existing Work
RQ1: How does integrating LoRA and instruction tuning affect FM performance in code generation?	LoRA enabled fine-tuning only 0.47% of parameters, improving TypeScript Pass@1 scores from 29.6% to 37.9% (base) and from 39% to 42% (instruct). Instruction tuning also supported multitask learning (e.g., code summarization).	Sec. 4.2	Benchmarked on the BigCode Leaderboard; performance competitive with top open models like StarCoder and Code Llama.
RQ2: Can the proposed approach meet domain-specific needs, particularly for TypeScript-based applications?	The specialized model generated accurate, project-specific code in private repositories—something general-purpose models like ChatGPT failed to do—while preserving data confidentiality.	Sec. 4.3	Few existing studies focus on domain-specific code generation for private TypeScript projects; this work addresses a largely unexplored area.
RQ3: What are the cost and resource benefits compared to traditional FM training?	LoRA-based fine-tuning reduced training time to under one hour on 13B models, avoiding full model retraining and enabling efficient multi-task adaptation with minimal resources.	Sec. 4.1 and 4.2	Aligns with recent research on parameter-efficient fine-tuning. LoRA remains the most widely adopted and practical approach.

foundational model by producing code snippets that closely align with the source code of personal projects. However, due to the limited size of the private instruction dataset (consisting of 40 samples), this model occasionally generates irrelevant or mismatched results. Notably, it avoids excessive bias towards the original model and frequently generates project-relevant outcomes. Additionally, both proposed models demonstrate rapid inference capabilities using vFM [39], reducing the computation time substantially compared to other models encountered during the research. Success has been achieved in generating code based on project data while maintaining the confidentiality of personal source code, along with combining code generation and summary explanations within a single model through instruction-tuning. Furthermore, the original model's parameter count has been preserved, with a training time of approximately one hour, facilitated by the

Low-Rank Adaptation (LoRA) parameter tuning method. Finally, the development of the *TypeScript-Instruct 20K* dataset stands as a significant contribution toward enhancing future TypeScript code generation models within the scope of this paper. In summary, the two proposed models have effectively addressed the primary objectives of achieving a "holy grail" in code generation and advancing the capability of understanding specific personal projects.

5.2. Limitations

Despite promising findings, this study faces several limitations that raise threats to the validity of its conclusions. We organize the discussion of these limitations according to four commonly used validity categories: construct, internal, external, and reliability validity [49, 47, 46].

5.2.1. Construct Validity

Construct validity refers to how well the experiments reflect the concepts being studied. We used the common benchmarks such as the Multi-Lingual HumanEval dataset for measuring code generation performance, and the most popular metrics for evaluating performance of FMs Pass@k [50, 51, 52, 53, 54, 55]. However, we are aware that the metrics do not fully capture dimensions like code readability, maintainability, or execution correctness beyond test cases. Although we manually assessed code quality and explanation capabilities, more rigorous human-in-the-loop evaluations (e.g., professional developer ratings) could further substantiate the constructs being measured.

5.2.2. Internal Validity

Internal validity concerns the degree to which the observed results can be attributed to the experimental treatment rather than confounding variables. Our experiments clearly showed performance gains when applying Low-Rank Adaptation (LoRA) and instruction tuning; however, we could not fully isolate the effects of dataset composition or preprocessing quality on model behavior. In particular, the CodeLSI Specialized Knowledge Model was trained on a relatively small dataset (40 samples), introducing a potential confounding factor—performance improvements may partially stem from dataset-specific patterns rather than true generalization to private codebases.

5.2.3. External Validity

External validity refers to the generalizability of results beyond the experimental settings. Our study focused exclusively on the TypeScript programming language. While this choice addresses a significant gap in current FMs, it limits applicability to other widely used programming languages (e.g., Java, C). Similarly, the use of private, domain-specific data for fine-tuning improves relevance for individual projects, but raises questions about

whether the same results can be replicated across diverse domains or team workflows. Expanding experimentation to multilingual and cross-domain scenarios would enhance generalizability.

5.2.4. Reliability

Reliability concerns the consistency and dependability of the results obtained. In this study, we observed consistent performance improvements across both benchmark evaluations and qualitative use cases, indicating stable model behavior. However, the relatively small domain-specific dataset (40 instruction samples) limits the robustness of these findings. Additionally, the absence of formal statistical analysis—such as hypothesis testing or confidence intervals—reduces confidence in the generalizability of the observed improvements. Nonetheless, the reliability of our experimental procedures is strengthened by the open availability of our models, datasets, and training configurations on Hugging Face, which facilitates reproducibility and independent verification.

6. Conclusion

This study presents a practical and efficient approach to domain-specific code generation through the development of the CodeLSI framework, leveraging Low-Rank Adaptation (LoRA) and instruction tuning. Our empirical results demonstrate that CodeLSI significantly improves the performance of TypeScript code generation tasks, outperforming baseline models such as CodeLlama-Instruct while maintaining computational efficiency. In particular, the CodeLSI General Knowledge Model showed substantial improvements in benchmark accuracy, and the Specialized Knowledge Model was capable of generating accurate, project-specific code within private repositories—an area largely unaddressed in prior work. The research also contributes a new, publicly available TypeScript-Instruct 20K dataset and shows that parameter-efficient methods like LoRA can enable rapid fine-tuning using limited resources, making specialized FM adaptation more accessible. Despite these contributions, several open challenges remain. Future work will focus on:

- Expanding language support beyond TypeScript to improve generalizability across software ecosystems.
- Enhancing syntactic robustness to minimize reliance on post-processing, ensuring that generated code is consistently executable and standards-compliant.
- Automating instruction dataset creation by extracting meaningful prompts and responses from existing codebases, especially using comments and docstrings as input signals.
- Broadening evaluation scope, including comparisons against a wider range of recent open and proprietary models, and adopting new fine-tuning strategies such as adapter-based tuning or Mixture-of-Experts models.

- Scaling empirical validation through more diverse datasets, programming languages, and task types, paired with formal statistical analyses to strengthen generalizability and reliability.

As FMs continue to evolve rapidly, ongoing benchmarking and methodological refinement will be essential. The insights and tools developed in this work provide a strong foundation for further research into specialized, efficient, and trustworthy code generation systems.

References

[1] F. J. Budinsky, M. A. Finnie, J. M. Vlissides, and P. S. Yu. Automatic code generation from design patterns. *IBM Systems Journal*, 35(2):151–171, 1996.

[2] Jack Herrington. *Code Generation in Action*. Manning Publications Co., USA, June 2003.

[3] Enrique Dehaerne, Bappaditya Dey, Sandip Halder, Stefan De Gendt, and Wannes Meert. Code Generation Using Machine Learning: A Systematic Review. *IEEE Access*, 10:82434–82455, 2022.

[4] Zohar Manna and Richard J. Waldinger. Toward automatic program synthesis. *Communications of the ACM*, 14(3):151–165, 1971.

[5] Sumit Gulwani, Oleksandr Polozov, and Rishabh Singh. Program synthesis. *Foundations and Trends® in Programming Languages*, 4(1-2):1–119, 2017.

[6] Richard Soley and OMG Group. Model driven architecture. *OMG White Paper*, 308(308):5, 2000.

[7] Burak Uzun and Bedir Tekinerdogan. Model-driven architecture based testing: A systematic literature review. *Information and Software Technology*, 102:30–48, 2018.

[8] Eugene Syriani, Lechanceux Luhunu, and Houari Sahraoui. Systematic mapping study of template-based code generation. *Computer Languages, Systems and Structures*, 52:43–62, 2018.

[9] Heling Cao, YangXia Meng, Jianshu Shi, Lei Li, Tiaoli Liao, and Chenyang Zhao. A survey on automatic bug fixing. In *Proceedings of 2020 6th International Symposium on System and Software Reliability (ISSSR)*, pages 122–131. IEEE, 2020.

[10] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 1(8):9, 2019.

[11] Fang Liu, Yang Liu, Lin Shi, Houkun Huang, Ruifeng Wang, Zhen Yang, Li Zhang, Zhongqi Li, and Yuchi Ma. Exploring and Evaluating Hallucinations in LLM-Powered Code Generation, May 2024. arXiv:2404.00971 [cs].

[12] Jianxun Wang and Yixiang Chen. A Review on Code Generation with LLMs: Application and Evaluation. In *2023 IEEE International Conference on Medical Artificial Intelligence (MedAI)*, pages 284–289, November 2023.

[13] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. A Survey on Large Language Models for Code Generation, November 2024. arXiv:2406.00515 [cs].

[14] Sathvik Joel, Jie JW Wu, and Fatemeh H. Fard. A Survey on LLM-based Code Generation for Low-Resource and Domain-Specific Programming Languages, November 2024. arXiv:2410.03981 [cs].

[15] Zhikai Chen, Haitao Mao, Hang Li, Wei Jin, Hongzhi Wen, Xiaochi Wei, Shuaiqiang Wang, Dawei Yin, Wenqi Fan, Hiu Liu, and Jiliang Tang. Exploring the potential of large language models (LLMs) in learning on graphs. *ACM SIGKDD Explorations Newsletter*, 25(2):42–61, 2024.

[16] Vladislav Lialin, Vijeta Deshpande, and Anna Rumshisky. Scaling down to scale up: A guide to parameter-efficient fine-tuning. *arXiv preprint arXiv:2303.15647*, 2023.

[17] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LORA: Low-Rank Adaptation of Large Language Models. *arXiv preprint arXiv:2106.09685*, 2021.

[18] Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V. Le. Finetuned Language Models Are Zero-Shot Learners. *arXiv preprint arXiv:2109.01652*, 2022.

[19] Shayne Longpre, Le Hou, Tu Vu, Albert Webson, Hyung Won Chung, Yi Tay, Denny Zhou, Quoc V Le, Barret Zoph, Jason Wei, Adam Roberts, Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett. The Flan Collection: Designing Data and Methods for Effective Instruction Tuning. In *Proceedings of 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 22631–22648. PMLR, 23–29 Jul 2023.

[20] Christopher D Manning and Hinrich Schütze. *Foundations of Statistical Natural Language Processing*. MIT Press, 1999.

[21] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in neural information processing systems*, pages 2672–2680, 2014.

[22] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

[23] Jeremy Howard and Sebastian Ruder. Universal language model fine-tuning for text classification. *arXiv preprint arXiv:1801.06146*, 2018.

[24] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*, 2020.

[25] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv preprint arXiv:1810.04805*, 2019.

[26] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. PaLM: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023.

[27] Yuchao Gu, Xintao Wang, Jay Zhangjie Wu, Yujun Shi, Yunpeng Chen, Zihan Fan, Wuyou Xiao, Rui Zhao, Shuning Chang, Weijia Wu, Yixiao Ge, Ying Shan, and Mike Zheng Shou. Mix-of-Show: Decentralized Low-Rank Adaptation for Multi-Concept Customization of Diffusion Models. In *Proceedings of Advances in Neural Information Processing Systems*, volume 36, pages 15890–15902. Curran Associates, Inc., 2023.

[28] Shayne Longpre, Le Hou, Tu Vu, Albert Webson, Hyung Won Chung, Yi Tay, Denny Zhou, Quoc V Le, Barret Zoph, Jason Wei, Adam Roberts, Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett. The Flan Collection: Designing Data and Methods for Effective Instruction Tuning. In *40th International Conference on Machine Learning*, Machine Learning Research, pages 22631–22648. PMLR, 23–29 Jul 2023.

[29] Charles Rich and Richard C. Waters. Automatic programming: myths and prospects. *Computer*, 21(8):40–51, 1988.

[30] Eric Schkufza, Rahul Sharma, and Alex Aiken. Stochastic superoptimization. *ACM SIGARCH Computer Architecture News*, 41(1):305–316, 2013.

[31] Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. *ACM Sigplan Notices*, 46(1):317–330, 2011.

[32] Oleksandr Polozov and Sumit Gulwani. Flashmeta: A framework for inductive program synthesis. In *Proceedings of 2015 ACM SIGPLAN International Conference on*

Object-Oriented Programming, Systems, Languages, and Applications, pages 107–126, 2015.

[33] Staphord Bengesi, Hoda El-Sayed, Md Kamruzzaman Sarker, Yao Houkpati, John Irungu, and Timothy Oladunni. Advancements in Generative AI: A Comprehensive Review of GANs, GPT, Autoencoders, Diffusion Model, and Transformers. *IEEE Access*, 2024.

[34] Sanjay Kukreja, Tarun Kumar, Amit Purohit, Abhijit Dasgupta, and Debashis Guha. A literature survey on open source large language models. In *Proceedings of 2024 7th International Conference on Computers in Management and Business*, pages 133–143, 2024.

[35] Yangxiao Cai, Peng Liang, Yifei Wang, Zengyang Li, and Mojtaba Shahin. Demystifying issues, causes and solutions in LLM open-source projects. *Journal of Systems and Software*, 227:112452, September 2025.

[36] Toufique Ahmed, Christian Bird, Premkumar Devanbu, and Saikat Chakraborty. Studying LLM Performance on Closed- and Open-source Data, February 2024. arXiv:2402.15100 [cs].

[37] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, Qian Liu, Evgenii Zheltonozhskii, Terry Yue Zhuo, Thomas Wang, Olivier Dehaene, Mishig Davaadorj, Joel Lamy-Poirier, João Monteiro, Oleh Shliazhko, Nicolas Gontier, Nicholas Meade, Armel Zebaze, Ming-Ho Yee, Logesh Kumar Umapathi, Jian Zhu, Benjamin Lipkin, Muhtasham Oblokulov, Zhiruo Wang, Rudra Murthy, Jason Stillerman, Siva Sankalp Patel, Dmitry Abulkhanov, Marco Zocca, Manan Dey, Zhihan Zhang, Nour Fahmy, Urvashi Bhattacharyya, Wenhao Yu, Swayam Singh, Sasha Luccioni, Paulo Villegas, Maxim Kunakov, Fedor Zhdanov, Manuel Romero, Tony Lee, Nadav Timor, Jennifer Ding, Claire Schlesinger, Hailey Schoelkopf, Jan Ebert, Tri Dao, Mayank Mishra, Alex Gu, Jennifer Robinson, Carolyn Jane Anderson, Brendan Dolan-Gavitt, Danish Contractor, Siva Reddy, Daniel Fried, Dzmitry Bahdanau, Yacine Jernite, Carlos Muñoz Ferrandis, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. StarCoder: May the source be with you! *arXiv preprint arXiv:2305.06161*, 2023.

[38] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, and Yuxiong He. DeepSpeed Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale. *arXiv preprint arXiv:2207.00032*, 2022.

[39] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management

for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.

[40] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q. Feldman, Arjun Guha, Michael Greenberg, and Abhinav Jangda. MultiPL-E: A Scalable and Extensible Approach to Benchmarking Neural Code Generation. *arXiv preprint arXiv:2208.08227*, 2022.

[41] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*, 2021.

[42] Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, Dzmitry Bahdanau, Leandro von Werra, and Harm de Vries. The Stack: 3 TB of permissively licensed source code. *arXiv preprint arXiv:2211.15533*, 2022.

[43] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-Instruct: Aligning Language Models with Self-Generated Instructions. *arXiv preprint arXiv:2212.10560*, 2023.

[44] Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, and Daxin Jiang. WizardLM: Empowering Large Language Models to Follow Complex Instructions. *arXiv preprint arXiv:2304.12244*, 2023.

[45] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. Code Llama: Open Foundation Models for Code. *arXiv preprint arXiv:2308.12950*, 2023.

[46] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. Experiments. In Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén, editors, *Experimentation in Software Engineering*, pages 73–83. Springer, Berlin, Heidelberg, 2024.

[47] B.A. Kitchenham, S.L. Pfleeger, L.M. Pickard, P.W. Jones, D.C. Hoaglin, K. El Emam, and J. Rosenberg. Preliminary guidelines for empirical research in software engineering. *IEEE Transactions on Software Engineering*, 28(8):721–734, August 2002.

[48] Le Vu Minh Huy. TypeScript Instruct Public Model 13B - mhhmm/typescript-instruct-20k-v2. Available at: https://huggingface.co/mhhmm/typescript-instruct-20k-v2, 2023.

[49] Per Runeson and Martin Höst. Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2):131, December 2008. Number: 2.

[50] Tao Huang, Zhihong Sun, Zhi Jin, Ge Li, and Chen Lyu. Knowledge-Aware Code Generation with Large Language Models. In *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension*, ICPC '24, pages 52–63, New York, NY, USA, June 2024. Association for Computing Machinery.

[51] Zeeshan Rasheed, Malik Abdul Sami, Kai-Kristian Kemell, Muhammad Waseem, Mika Saari, Kari Systä, and Pekka Abrahamsson. CodePori: Large-Scale System for Autonomous Software Development Using Multi-Agent Technology, September 2024. arXiv:2402.01411 [cs].

[52] Yunda Tsai, Mingjie Liu, and Haoxing Ren. RTLFixer: Automatically Fixing RTL Syntax Errors with Large Language Model. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*, DAC '24, pages 1–6, New York, NY, USA, November 2024. Association for Computing Machinery.

[53] Sangyeop Yeo, Yu-Seung Ma, Sang Cheol Kim, Hyungkook Jun, and Taeho Kim. Framework for evaluating code generation ability of large language models. *ETRI Journal*, 46(1):106–117, 2024. _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.4218/etrij.2023-0357.

[54] Bardia Nadimi, Ghali Omar Boutaib, and Hao Zheng. VeriMind: Agentic LLM for Automated Verilog Generation with a Novel Evaluation Metric, April 2025. arXiv:2503.16514 [cs].

[55] Debalina Ghosh Paul, Hong Zhu, and Ian Bayley. Benchmarks and Metrics for Evaluations of Code Generation: A Critical Review. In *2024 IEEE International Conference on Artificial Intelligence Testing (AITest)*, pages 87–94, July 2024. ISSN: 2835-3560.